

Computer Organisation & Program Execution 2019

Digital Logic

Uwe R. Zimmer - The Australian National University

Digital Logic

Boolean Values & Operators

There are two values:

e.g. **True** and **False**.
 Two binary operators on expressions a, b :

$a \vee b$ (aka $a + b$ or "a OR b" or SUM)
 $a \wedge b$ (aka $a \cdot b$ or "a AND b" or PRODUCT)

One unary operator on an expression a :

$\bar{a}$ (aka $\neg a$ or a' or "NOT a ")

Truth tables:

a	b	$a \vee b$	$a \wedge b$	$\bar{a}$
False	False	False	False	True
False	True	True	False	True
True	False	True	False	False
True	True	True	True	False

Digital Logic

References for this chapter

[Patterson 17]
 David A. Patterson & John L. Hennessy
 Computer Organization and Design – The Hardware/Software Interface
 Appendix A "The Basics of Logic Design"
 ARM edition, Morgan Kaufmann 2017

Digital Logic

Axiomatic Boolean Algebra (Whitehead 1898)

$\vee$ -Laws	$\wedge$ -Laws
$a \vee a = a$	$a \wedge a = a$
$a \vee b = b \vee a$	$a \wedge b = b \wedge a$
$(a \vee b) \vee c = a \vee (b \vee c)$	$(a \wedge b) \wedge c = a \wedge (b \wedge c)$
$a \vee (a \wedge b) = a$	$a \wedge (a \vee b) = a$
$a \wedge (b \vee c) = (a \wedge b) \vee (a \wedge c)$	$a \vee (b \wedge c) = (a \vee b) \wedge (a \vee c)$
$a \vee \text{True} = a$	$a \wedge \text{False} = a$
$a \vee \text{False} = a$	$a \wedge \text{True} = a$
$a \vee \bar{a} = \text{True}$	$a \wedge \bar{a} = \text{False}$
$a \vee \bar{a} = \text{True}$	$a \wedge \bar{a} = \text{False}$

Algebras allow for easier reasoning than truth tables.

Digital Logic

It starts with a thought ...

An Investigation of the Laws of Thought on Which are Founded the Mathematical Theories of Logic and Probabilities
 by **George Boole**, 1854

Digital Logic

Axiomatic Boolean Algebra (Huntington 1904)

$\vee$ -Laws	$\wedge$ -Laws
$a \vee b = b \vee a$	$a \wedge b = b \wedge a$
$(a \vee b) \vee c = a \vee (b \vee c)$	$(a \wedge b) \wedge c = a \wedge (b \wedge c)$
$a \vee (a \wedge b) = a$	$a \wedge (a \vee b) = a$
$a \wedge (b \vee c) = (a \wedge b) \vee (a \wedge c)$	$a \vee (b \wedge c) = (a \vee b) \wedge (a \vee c)$
$a \vee \text{True} = a$	$a \wedge \text{False} = a$
$a \vee \text{False} = a$	$a \wedge \text{True} = a$
$a \vee \bar{a} = \text{True}$	$a \wedge \bar{a} = \text{False}$
$a \vee \bar{a} = \text{True}$	$a \wedge \bar{a} = \text{False}$

Digital Logic

Axiomatic Boolean Algebra

... many other axiomatic formulations of Boolean algebra exist.

Digital Logic

Redundant Boolean Algebra

$\vee$ -Laws	$\wedge$ -Laws
$a \vee a = a$	$a \wedge a = a$
$a \vee b = b \vee a$	$a \wedge b = b \wedge a$
$(a \vee b) \vee c = a \vee (b \vee c)$	$(a \wedge b) \wedge c = a \wedge (b \wedge c)$
$a \vee (a \wedge b) = a$	$a \wedge (a \vee b) = a$
$a \vee (b \wedge c) = (a \vee b) \wedge (a \vee c)$	$a \wedge (b \vee c) = (a \wedge b) \vee (a \wedge c)$
$a \vee \text{True} = a$	$a \wedge \text{False} = a$
$a \vee \text{False} = a$	$a \wedge \text{True} = a$
$a \vee \bar{a} = \text{True}$	$a \wedge \bar{a} = \text{False}$
$a \vee \bar{a} = \text{True}$	$a \wedge \bar{a} = \text{False}$

$\bar{\bar{a}} = a$

Digital Logic

Common Boolean operators

Commonly used operators on expressions a, b to define boolean algebras:

$a \vee b$ (aka $a + b$ or "a OR b" or SUM)
 $a \wedge b$ (aka $a \cdot b$ or "a AND b" or PRODUCT)
 $\bar{a}$ (aka $\neg a$ or a' or "not a ")

Other handy operators:

$a \rightarrow b = (\bar{a} \vee b)$ (aka "a IMPLIES b")
 $(a = b) = (a \wedge b) \vee (\bar{a} \wedge \bar{b})$ (aka "a EQUALS b")
 $a \oplus b = (a \wedge \bar{b}) \vee (\bar{a} \wedge b)$ (aka "a EXCLUSIVE-OR b" or "a XOR b")
 $a \wedge \bar{b} = (\bar{a} \vee \bar{b})$ (aka "a NOT-AND b" or "a NAND b")
 $a \vee \bar{b} = (\bar{a} \wedge \bar{b})$ (aka "a NOT-OR b" or "a NOR b")

NAND and NOR are the only sole sufficient boolean operators, i.e. you can reduce any boolean expression to only NAND or only NOR operators.

Digital Logic

All binary Boolean operators

Inputs a b	Function	Name	Sum of products	NAND build $\neg, \wedge, \vee, \neg$	Don't cares
a F b F	False	Constant FALSE			
a F b T	$a \wedge b$	AND		$a \wedge b$	a, b
a T b F	$a \rightarrow b$	NOT-IMPLICATION	$(a \wedge \neg b)$		
a T b T	$a \rightarrow b$	NOT-IMPLICATION	$(\neg a \wedge b)$		b
a F b F	$a \oplus b$	EXCLUSIVE-OR			
a F b T	$a \oplus b$	EXCLUSIVE-OR	$(a \wedge \neg b) \vee (\neg a \wedge b)$		a
a T b F	$a \oplus b$	EXCLUSIVE-OR			
a T b T	$a \oplus b$	EXCLUSIVE-OR		$\overline{a \wedge b \wedge b}$	
a F b F	$a \vee b$	OR			
a F b T	$a \vee b$	OR	$(\neg a \wedge b)$		
a T b F	$a \vee b$	OR	$(a \wedge \neg b)$		
a T b T	$a \vee b$	OR	$(a \wedge b) \vee (\neg a \wedge b)$		a
a F b F	$a \rightarrow b$	IMPLICATION			
a F b T	$a \rightarrow b$	IMPLICATION	$a \vee \neg b$		
a T b F	$a \rightarrow b$	IMPLICATION		$\overline{a \wedge a}$	b
a T b T	$a \rightarrow b$	IMPLICATION			
a F b F	$a \rightarrow b$	IMPLICATION			
a F b T	$a \rightarrow b$	IMPLICATION			
a T b F	$a \rightarrow b$	IMPLICATION			
a T b T	$a \rightarrow b$	IMPLICATION			
a F b F	True	Constant True			a, b

Digital Logic

Combinational Logic Functions

Logic is reducible/equivalent to pure functions: there are no states!

If the function is combinational then there is only one output for any combination of inputs, e.g. the function can be written out as a truth table:

a b c	Output q	minterms	Simplified minterms
F F F	F	$\neg a \wedge \neg b \wedge \neg c$	
F F T	F	$\neg a \wedge \neg b \wedge c$	
F T F	F	$\neg a \wedge b \wedge \neg c$	
F T T	F	$\neg a \wedge b \wedge c$	$b \wedge \neg c$
T F F	T	$a \wedge \neg b \wedge \neg c$	$a \wedge \neg b$
T F T	T	$a \wedge \neg b \wedge c$	
T T F	T	$a \wedge b \wedge \neg c$	
T T T	T	$a \wedge b \wedge c$	

Sum of minterms: $q = (\neg a \wedge \neg b \wedge \neg c) \vee (\neg a \wedge \neg b \wedge c) \vee (a \wedge \neg b \wedge \neg c) \vee (a \wedge \neg b \wedge c) \vee (a \wedge b \wedge \neg c) \vee (a \wedge b \wedge c)$

Sum of simplified minterms: $q = (a \wedge \neg b) \vee (b \wedge \neg c)$

Simplifications can be done by (automated) algebraic transformations, Karnaugh maps or others

Digital Logic

Combinational Logic Functions

Logic is reducible/equivalent to pure functions: there are no states!

If the function is combinational then there is only one output for any combination of inputs, e.g. the function can be written out as a truth table:

a b c	Output q	minterms	Simplified minterms
F F F	F	$\neg a \wedge \neg b \wedge \neg c$	
F F T	F	$\neg a \wedge \neg b \wedge c$	
F T F	T	$\neg a \wedge b \wedge \neg c$	
F T T	T	$\neg a \wedge b \wedge c$	$\neg a \wedge b$
T F F	T	$a \wedge \neg b \wedge \neg c$	
T F T	T	$a \wedge \neg b \wedge c$	
T T F	T	$a \wedge b \wedge \neg c$	
T T T	T	$a \wedge b \wedge c$	

maxterms product $q = (a \vee b \vee c) \wedge (a \vee b \vee \neg c) \wedge (a \vee \neg b \vee \neg c) \wedge (a \vee \neg b \vee c)$

Digital Logic

Combinational Logic Functions

Logic is reducible/equivalent to pure functions: there are no states!

If the function is combinational then there is only one output for any combination of inputs, e.g. the function can be written out as a truth table:

a b c	Output q	minterms	Simplified minterms
F F F	F	$\neg a \wedge \neg b \wedge \neg c$	
F F T	F	$\neg a \wedge \neg b \wedge c$	
F T F	F	$\neg a \wedge b \wedge \neg c$	
F T T	F	$\neg a \wedge b \wedge c$	
T F F	T	$a \wedge \neg b \wedge \neg c$	$a \wedge \neg b$
T F T	T	$a \wedge \neg b \wedge c$	
T T F	T	$a \wedge b \wedge \neg c$	
T T T	T	$a \wedge b \wedge c$	

Digital Logic

Combinational Logic Functions

Logic is reducible/equivalent to pure functions: there are no states!

If the function is combinational then there is only one output for any combination of inputs, e.g. the function can be written out as a truth table:

a b c	Output q	minterms	Simplified minterms
F F F	F	$\neg a \wedge \neg b \wedge \neg c$	
F F T	F	$\neg a \wedge \neg b \wedge c$	
F T F	T	$\neg a \wedge b \wedge \neg c$	
F T T	T	$\neg a \wedge b \wedge c$	$b \wedge \neg c$
T F F	T	$a \wedge \neg b \wedge \neg c$	$a \wedge \neg b$
T F T	T	$a \wedge \neg b \wedge c$	
T T F	T	$a \wedge b \wedge \neg c$	
T T T	T	$a \wedge b \wedge c$	

Sum of minterms: $q = (\neg a \wedge b \wedge \neg c) \vee (\neg a \wedge b \wedge c) \vee (a \wedge \neg b \wedge \neg c) \vee (a \wedge \neg b \wedge c) \vee (a \wedge b \wedge \neg c) \vee (a \wedge b \wedge c)$

Sum of simplified minterms: $q = (a \wedge \neg b) \vee (b \wedge \neg c)$

Simplifications can be done by (automated) algebraic transformations, Karnaugh maps or others

Digital Logic

Combinational Logic Functions

Logic is reducible/equivalent to pure functions: there are no states!

If the function is combinational then there is only one output for any combination of inputs, e.g. the function can be written out as a truth table:

a b c	Output q	minterms	Simplified minterms
F F F	F	$\neg a \wedge \neg b \wedge \neg c$	
F F T	F	$\neg a \wedge \neg b \wedge c$	
F T F	T	$\neg a \wedge b \wedge \neg c$	
F T T	T	$\neg a \wedge b \wedge c$	$\neg a \wedge b$
T F F	T	$a \wedge \neg b \wedge \neg c$	
T F T	T	$a \wedge \neg b \wedge c$	
T T F	T	$a \wedge b \wedge \neg c$	
T T T	T	$a \wedge b \wedge c$	

maxterms product $q = (a \vee b \vee c) \wedge (a \vee b \vee \neg c) \wedge (a \vee \neg b \vee \neg c) \wedge (a \vee \neg b \vee c)$

simplified maxterms product $q = (a \vee b) \wedge (b \vee \neg c)$

Simplifications can be done by (automated) algebraic transformations, Karnaugh maps or others

Digital Logic

Combinational Logic Functions

Logic is reducible/equivalent to pure functions: there are no states!

If the function is combinational then there is only one output for any combination of inputs, e.g. the function can be written out as a truth table:

a b c	Output q	minterms	Simplified minterms
F F F	F	$\neg a \wedge \neg b \wedge \neg c$	
F F T	F	$\neg a \wedge \neg b \wedge c$	
F T F	T	$\neg a \wedge b \wedge \neg c$	
F T T	T	$\neg a \wedge b \wedge c$	
T F F	T	$a \wedge \neg b \wedge \neg c$	$a \wedge \neg b$
T F T	T	$a \wedge \neg b \wedge c$	
T T F	T	$a \wedge b \wedge \neg c$	
T T T	T	$a \wedge b \wedge c$	

Sum of minterms: $q = (\neg a \wedge b \wedge \neg c) \vee (\neg a \wedge b \wedge c) \vee (a \wedge \neg b \wedge \neg c) \vee (a \wedge \neg b \wedge c) \vee (a \wedge b \wedge \neg c) \vee (a \wedge b \wedge c)$

Digital Logic

Combinational Logic Functions

Logic is reducible/equivalent to pure functions: there are no states!

If the function is combinational then there is only one output for any combination of inputs, e.g. the function can be written out as a truth table:

a b c	Output q	minterms	Simplified minterms
F F F	F	$\neg a \wedge \neg b \wedge \neg c$	
F F T	F	$\neg a \wedge \neg b \wedge c$	
F T F	F	$\neg a \wedge b \wedge \neg c$	
F T T	F	$\neg a \wedge b \wedge c$	
T F F	T	$a \wedge \neg b \wedge \neg c$	
T F T	T	$a \wedge \neg b \wedge c$	
T T F	T	$a \wedge b \wedge \neg c$	
T T T	T	$a \wedge b \wedge c$	

Digital Logic

Combinational Logic Functions

Logic is reducible/equivalent to pure functions: there are no states!

If the function is combinational then there is only one output for any combination of inputs, e.g. the function can be written out as a truth table:

a b c	Output q	minterms	Simplified minterms
F F F	F	$\neg a \wedge \neg b \wedge \neg c$	
F F T	F	$\neg a \wedge \neg b \wedge c$	
F T F	T	$\neg a \wedge b \wedge \neg c$	
F T T	T	$\neg a \wedge b \wedge c$	
T F F	T	$a \wedge \neg b \wedge \neg c$	
T F T	T	$a \wedge \neg b \wedge c$	
T T F	T	$a \wedge b \wedge \neg c$	
T T T	T	$a \wedge b \wedge c$	

maxterms product $q = (a \vee b \vee c) \wedge (a \vee b \vee \neg c) \wedge (a \vee \neg b \vee \neg c) \wedge (a \vee \neg b \vee c)$

simplified maxterms product $q = (a \vee b) \wedge (b \vee \neg c)$

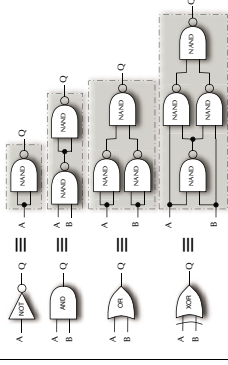
Simplifications can be done by (automated) algebraic transformations, Karnaugh maps or others

Digital Electronics

Symbolic: $\equiv$  $\equiv$ $\overline{A \cdot B}$

Diagram: 

Technology: 



Digital Logic

Combinational Logic Functions

⚡ The logic equivalent to pure functions: there are no states!

If the function is combinational then there is only one output for any combination of inputs, e.g. the function can be written out as a truth table:

a	b	c	Output q	Minterms	Simplified minterms
F	F	F	F	$\overline{a} \wedge \overline{b} \wedge \overline{c}$	$\overline{a} \wedge \overline{b}$
F	F	T	F	$\overline{a} \wedge \overline{b} \wedge c$	$\overline{a} \wedge \overline{b}$
F	T	F	F	$\overline{a} \wedge b \wedge \overline{c}$	$\overline{a} \wedge b$
F	T	T	F	$\overline{a} \wedge b \wedge c$	$\overline{a} \wedge b$
T	F	F	T	$a \wedge \overline{b} \wedge \overline{c}$	$a \wedge \overline{b}$
T	F	T	T	$a \wedge \overline{b} \wedge c$	$a \wedge \overline{b}$
T	T	F	T	$a \wedge b \wedge \overline{c}$	$a \wedge b$
T	T	T	T	$a \wedge b \wedge c$	$a \wedge b$

Sum of minterms: $q = (\overline{a} \wedge \overline{b} \wedge \overline{c}) \vee (\overline{a} \wedge \overline{b} \wedge c) \vee (\overline{a} \wedge b \wedge \overline{c}) \vee (\overline{a} \wedge b \wedge c) \vee (a \wedge \overline{b} \wedge \overline{c}) \vee (a \wedge \overline{b} \wedge c) \vee (a \wedge b \wedge \overline{c}) \vee (a \wedge b \wedge c)$

Sum of simplified minterms: $q = (\overline{a} \wedge \overline{b}) \vee (\overline{a} \wedge b) \vee (a \wedge \overline{b}) \vee (a \wedge b)$

Simplifications can be done by (automated) algebraic transformations, Karnaugh maps or others

Digital Logic

Encoding

Assuming a type can have 7 different values, many forms of encoding are possible:

Enumeration type	Index	Value	Single bit	Gray code	Even parity	1-bit error detecting	1-bit error correcting	Binary
1	Secured	0000001	0000001	000	0000	0000000	0000000000	000
2	Taxi	0000010	0000010	001	0011	0000001	0000001111	001
3	Take-off	0000100	0000100	010	0101	0001100	0001100000	010
4	Cruising	0001000	0001000	011	0110	0111100	0001111111	011
5	Cliding	0010000	0010000	110	1001	0101010	1110000000	100
6	Approach	0100000	0100000	111	1010	1010101	1110001111	101
7	Landing	1000000	1000000	101	1100	1100110	1111110000	110

⚡ VHDL or Verilog gives you full control over the encoding.

Digital Logic

Combinational Logic Functions

⚡ The logic equivalent to pure functions: there are no states!

If the function is combinational then there is only one output for any combination of inputs, e.g. the function can be written out as a truth table:

a	b	c	Output q	Minterms	Simplified minterms
F	F	F	F	$\overline{a} \wedge \overline{b} \wedge \overline{c}$	$\overline{a} \wedge \overline{b}$
F	F	T	F	$\overline{a} \wedge \overline{b} \wedge c$	$\overline{a} \wedge \overline{b}$
F	T	F	F	$\overline{a} \wedge b \wedge \overline{c}$	$\overline{a} \wedge b$
F	T	T	F	$\overline{a} \wedge b \wedge c$	$\overline{a} \wedge b$
T	F	F	T	$a \wedge \overline{b} \wedge \overline{c}$	$a \wedge \overline{b}$
T	F	T	T	$a \wedge \overline{b} \wedge c$	$a \wedge \overline{b}$
T	T	F	T	$a \wedge b \wedge \overline{c}$	$a \wedge b$
T	T	T	T	$a \wedge b \wedge c$	$a \wedge b$

Sum of minterms: $q = (\overline{a} \wedge \overline{b} \wedge \overline{c}) \vee (\overline{a} \wedge \overline{b} \wedge c) \vee (\overline{a} \wedge b \wedge \overline{c}) \vee (\overline{a} \wedge b \wedge c) \vee (a \wedge \overline{b} \wedge \overline{c}) \vee (a \wedge \overline{b} \wedge c) \vee (a \wedge b \wedge \overline{c}) \vee (a \wedge b \wedge c)$

Sum of simplified minterms: $q = (\overline{a} \wedge \overline{b}) \vee (\overline{a} \wedge b) \vee (a \wedge \overline{b}) \vee (a \wedge b)$

Simplifications can be done by (automated) algebraic transformations, Karnaugh maps or others

Digital Logic

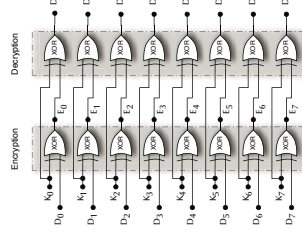
Processing Data

Encrypting a bit vector (whatever it represents) with a secret key:

Assuming the key is random and not used for anything else:

⚡ This is surprisingly secure

⚡ ... and extremely fast!



Digital Logic

Binary encoding

⚡ Encoding of choice if compactness is essential or you need to add values.

0	0	1	0	1	0	1	0	1	0	0
2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0			
↓	↓	↓	↓	↓	↓	↓	↓			
32	+	8	+	2						
										= 42

Digital Logic

Combinational Logic Functions

⚡ The logic equivalent to pure functions: there are no states!

If the function is combinational then there is only one output for any combination of inputs, e.g. the function can be written out as a truth table:

a	b	c	Output q	Minterms	Simplified minterms
F	F	F	F	$\overline{a} \wedge \overline{b} \wedge \overline{c}$	$\overline{a} \wedge \overline{b}$
F	F	T	F	$\overline{a} \wedge \overline{b} \wedge c$	$\overline{a} \wedge \overline{b}$
F	T	F	F	$\overline{a} \wedge b \wedge \overline{c}$	$\overline{a} \wedge b$
F	T	T	F	$\overline{a} \wedge b \wedge c$	$\overline{a} \wedge b$
T	F	F	T	$a \wedge \overline{b} \wedge \overline{c}$	$a \wedge \overline{b}$
T	F	T	T	$a \wedge \overline{b} \wedge c$	$a \wedge \overline{b}$
T	T	F	T	$a \wedge b \wedge \overline{c}$	$a \wedge b$
T	T	T	T	$a \wedge b \wedge c$	$a \wedge b$

Sum of minterms: $q = (\overline{a} \wedge \overline{b} \wedge \overline{c}) \vee (\overline{a} \wedge \overline{b} \wedge c) \vee (\overline{a} \wedge b \wedge \overline{c}) \vee (\overline{a} \wedge b \wedge c) \vee (a \wedge \overline{b} \wedge \overline{c}) \vee (a \wedge \overline{b} \wedge c) \vee (a \wedge b \wedge \overline{c}) \vee (a \wedge b \wedge c)$

Sum of simplified minterms: $q = (\overline{a} \wedge \overline{b}) \vee (\overline{a} \wedge b) \vee (a \wedge \overline{b}) \vee (a \wedge b)$

Simplifications can be done by (automated) algebraic transformations, Karnaugh maps or others

Digital Logic

Bit Vectors

Groups of bits could represent:

States, enumeration values, arrays of Booleans, numbers, etc, pp. ... or any grouping or combination of the above ⚡ **Algebraic Types**

The form of encoding could be chosen to optimize for:

- **Performance** ⚡ e.g. minimal decoding effort
- **Redundancy / error detection** ⚡ e.g. large Hamming distance
- **Safe transitions** ⚡ e.g. Gray codes
- **Physical mapping** ⚡ e.g. maps on existing hardware interfaces
- **Compactness** ⚡ e.g. holds the maximal number of values per memory cell

Digital Logic

Binary encoding

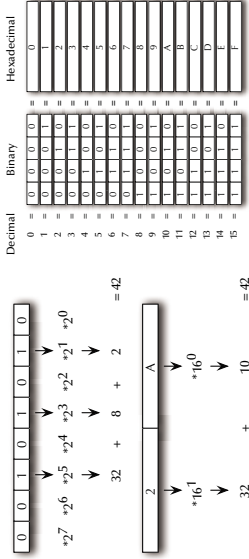
⚡ Encoding of choice if compactness is essential or you need to add values.

0	0	1	0	1	0	1	0	1	0	0
2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0			
↓	↓	↓	↓	↓	↓	↓	↓			
32	+	8	+	2						
										= 42

Digital Logic

Binary encoding

⚡ Encoding of choice: if compactness is essential or you need to add values.



Digital Logic

Half Adder

A	B	S	C	S minterms	C minterms
0	0	0	0	$\overline{A} \wedge \overline{B}$	$A \wedge B$
0	1	1	0	$\overline{A} \wedge B$	
1	0	1	0	$A \wedge \overline{B}$	
1	1	0	1		$A \wedge B$

Digital Logic

Half Adder

A	B	S	C	S minterms	C minterms
0	0	0	0	$\overline{A} \wedge \overline{B}$	$A \wedge B$
0	1	1	0	$\overline{A} \wedge B$	
1	0	1	0	$A \wedge \overline{B}$	
1	1	0	1		$A \wedge B$

$$\text{⚡ } S = (A \wedge \overline{B}) \vee (\overline{A} \wedge B)$$

$$\text{⚡ } C = A \wedge B$$

Digital Logic

Full Adder

A _j	B _j	C _{j-1}	S _j	C _j
0	0	0	0	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

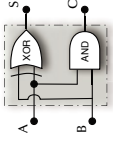
Digital Logic

Half Adder

A	B	S	C	S minterms	C minterms
0	0	0	0	$\overline{A} \wedge \overline{B}$	$A \wedge B$
0	1	1	0	$\overline{A} \wedge B$	
1	0	1	0	$A \wedge \overline{B}$	
1	1	0	1		$A \wedge B$

$$\text{⚡ } S = (A \wedge \overline{B}) \vee (\overline{A} \wedge B) = A \oplus B$$

$$\text{⚡ } C = A \wedge B$$



Digital Logic

Full Adder

A _j	B _j	C _{j-1}	S _j	C _j	S _j minterms	C _j minterms
0	0	0	0	0	$\overline{A}_j \wedge \overline{B}_j \wedge \overline{C}_{j-1}$	$A_j \wedge B_j \wedge \overline{C}_{j-1}$
0	1	0	1	0	$\overline{A}_j \wedge B_j \wedge \overline{C}_{j-1}$	$A_j \wedge \overline{B}_j \wedge \overline{C}_{j-1}$
0	1	1	0	1	$\overline{A}_j \wedge B_j \wedge C_{j-1}$	$\overline{A}_j \wedge \overline{B}_j \wedge C_{j-1}$
1	0	0	1	0	$A_j \wedge \overline{B}_j \wedge \overline{C}_{j-1}$	$A_j \wedge B_j \wedge \overline{C}_{j-1}$
1	0	1	0	1	$A_j \wedge \overline{B}_j \wedge C_{j-1}$	$\overline{A}_j \wedge B_j \wedge C_{j-1}$
1	1	0	0	1	$A_j \wedge B_j \wedge \overline{C}_{j-1}$	$A_j \wedge \overline{B}_j \wedge C_{j-1}$
1	1	1	1	1	$A_j \wedge B_j \wedge C_{j-1}$	$A_j \wedge B_j \wedge C_{j-1}$

$$\begin{aligned} \text{⚡ } S_j &= (A_j \wedge \overline{B}_j \wedge \overline{C}_{j-1}) \vee (\overline{A}_j \wedge B_j \wedge \overline{C}_{j-1}) \vee (\overline{A}_j \wedge \overline{B}_j \wedge C_{j-1}) \vee (A_j \wedge B_j \wedge C_{j-1}) \\ &= ((A_j \oplus B_j) \wedge \overline{C}_{j-1}) \vee ((\overline{A}_j \wedge \overline{B}_j) \vee (A_j \wedge B_j)) \wedge C_{j-1} \\ &= ((A_j \oplus B_j) \wedge \overline{C}_{j-1}) \vee ((A_j \oplus B_j) \wedge C_{j-1}) = (A_j \oplus B_j) \wedge C_{j-1} \\ &= (A_j \oplus B_j) \oplus C_{j-1} \end{aligned}$$

Digital Logic

Full Adder

A _j	B _j	C _{j-1}	S _j	C _j	S _j minterms	C _j minterms
0	0	0	0	0	$\overline{A}_j \wedge \overline{B}_j \wedge \overline{C}_{j-1}$	$A_j \wedge B_j \wedge \overline{C}_{j-1}$
0	1	0	1	0	$\overline{A}_j \wedge B_j \wedge \overline{C}_{j-1}$	$A_j \wedge \overline{B}_j \wedge \overline{C}_{j-1}$
0	1	1	0	1	$\overline{A}_j \wedge B_j \wedge C_{j-1}$	$\overline{A}_j \wedge \overline{B}_j \wedge C_{j-1}$
1	0	0	1	0	$A_j \wedge \overline{B}_j \wedge \overline{C}_{j-1}$	$A_j \wedge B_j \wedge \overline{C}_{j-1}$
1	0	1	0	1	$A_j \wedge \overline{B}_j \wedge C_{j-1}$	$\overline{A}_j \wedge B_j \wedge C_{j-1}$
1	1	0	0	1	$A_j \wedge B_j \wedge \overline{C}_{j-1}$	$A_j \wedge \overline{B}_j \wedge C_{j-1}$
1	1	1	1	1	$A_j \wedge B_j \wedge C_{j-1}$	$A_j \wedge B_j \wedge C_{j-1}$

$$\text{⚡ } S_j = (A_j \wedge \overline{B}_j \wedge \overline{C}_{j-1}) \vee (\overline{A}_j \wedge B_j \wedge \overline{C}_{j-1}) \vee (\overline{A}_j \wedge \overline{B}_j \wedge C_{j-1}) \vee (A_j \wedge B_j \wedge C_{j-1})$$

Digital Logic

Full Adder

A _j	B _j	C _{j-1}	S _j	C _j	S _j minterms	C _j minterms
0	0	0	0	0	$\overline{A}_j \wedge \overline{B}_j \wedge \overline{C}_{j-1}$	$A_j \wedge B_j \wedge \overline{C}_{j-1}$
0	1	0	1	0	$\overline{A}_j \wedge B_j \wedge \overline{C}_{j-1}$	$A_j \wedge \overline{B}_j \wedge \overline{C}_{j-1}$
0	1	1	0	1	$\overline{A}_j \wedge B_j \wedge C_{j-1}$	$\overline{A}_j \wedge \overline{B}_j \wedge C_{j-1}$
1	0	0	1	0	$A_j \wedge \overline{B}_j \wedge \overline{C}_{j-1}$	$A_j \wedge B_j \wedge \overline{C}_{j-1}$
1	0	1	0	1	$A_j \wedge \overline{B}_j \wedge C_{j-1}$	$\overline{A}_j \wedge B_j \wedge C_{j-1}$
1	1	0	0	1	$A_j \wedge B_j \wedge \overline{C}_{j-1}$	$A_j \wedge \overline{B}_j \wedge C_{j-1}$
1	1	1	1	1	$A_j \wedge B_j \wedge C_{j-1}$	$A_j \wedge B_j \wedge C_{j-1}$

$$\text{⚡ } S_j = (A_j \oplus B_j) \oplus C_{j-1}$$

$$\text{⚡ } C_j = (A_j \wedge B_j \wedge \overline{C}_{j-1}) \vee (\overline{A}_j \wedge B_j \wedge C_{j-1}) \vee (A_j \wedge \overline{B}_j \wedge C_{j-1}) \vee (A_j \wedge B_j \wedge C_{j-1})$$

Digital Logic

Full Adder

A_i	B_i	C_{i-1}	S_i	C_i	C_i minterms
0	0	0	0	0	$\overline{A_i} \wedge \overline{B_i} \wedge \overline{C_{i-1}}$
0	1	0	1	0	$\overline{A_i} \wedge B_i \wedge \overline{C_{i-1}}$
1	0	0	1	0	$A_i \wedge \overline{B_i} \wedge \overline{C_{i-1}}$
1	1	0	0	1	$A_i \wedge B_i \wedge \overline{C_{i-1}}$
0	0	1	0	1	$\overline{A_i} \wedge \overline{B_i} \wedge C_{i-1}$
0	1	1	1	0	$\overline{A_i} \wedge B_i \wedge C_{i-1}$
1	0	1	1	0	$A_i \wedge \overline{B_i} \wedge C_{i-1}$
1	1	1	1	1	$A_i \wedge B_i \wedge C_{i-1}$

$$S_i = (A_i \oplus B_i) \oplus C_{i-1}$$

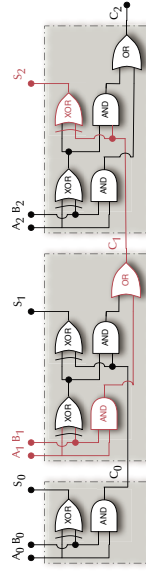
$$C_i = (A_i \wedge B_i \wedge \overline{C_{i-1}}) \vee (\overline{A_i} \wedge B_i \wedge C_{i-1}) \vee (A_i \wedge \overline{B_i} \wedge C_{i-1}) \vee (A_i \wedge B_i \wedge C_{i-1})$$

$$= (A_i \wedge B_i) \vee ((\overline{A_i} \wedge B_i) \vee (A_i \wedge \overline{B_i})) \wedge C_{i-1}$$

$$= (A_i \wedge B_i) \vee ((A_i \oplus B_i) \wedge C_{i-1})$$

Digital Logic

Ripple Carry Adder



$$2 + 2 = 4 !$$

Digital Logic

Radix complements

Can we define negative numbers such that our adder still works?

$$x - x = 0$$

Or: what can you add to 42 in an 8bit binary representation such that the result will be 2^8 (and hence 0 in 8bits)?

$$\begin{array}{r}
 000101010110 \\
 + \\
 11101011110 \\
 = \\
 100000000000
 \end{array}$$

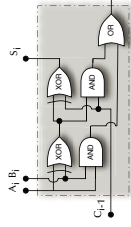
Digital Logic

Full Adder

A_i	B_i	C_{i-1}	S_i	C_i	C_i minterms
0	0	0	0	0	$\overline{A_i} \wedge \overline{B_i} \wedge \overline{C_{i-1}}$
0	1	0	1	0	$\overline{A_i} \wedge B_i \wedge \overline{C_{i-1}}$
1	0	0	1	0	$A_i \wedge \overline{B_i} \wedge \overline{C_{i-1}}$
1	1	0	0	1	$A_i \wedge B_i \wedge \overline{C_{i-1}}$
0	0	1	0	1	$\overline{A_i} \wedge \overline{B_i} \wedge C_{i-1}$
0	1	1	1	0	$\overline{A_i} \wedge B_i \wedge C_{i-1}$
1	0	1	1	0	$A_i \wedge \overline{B_i} \wedge C_{i-1}$
1	1	1	1	1	$A_i \wedge B_i \wedge C_{i-1}$

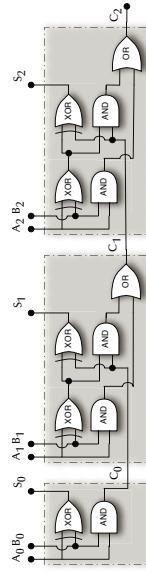
$$S_i = (A_i \oplus B_i) \oplus C_{i-1}$$

$$C_i = (A_i \wedge B_i) \vee ((A_i \oplus B_i) \wedge C_{i-1})$$



Digital Logic

Ripple Carry Adder



$$2 - 1 = 1 ?$$

Digital Logic

Radix complements

Can we define negative numbers such that our adder still works?

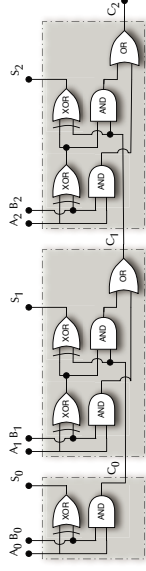
$$x - x = 0$$

Or: what can you add to 42 in an 8bit binary representation such that the result will be 2^8 (and hence 0 in 8bits)?

$$\begin{array}{r}
 000101010110 \\
 + \\
 11101011110 \\
 = \\
 100000000000
 \end{array}$$

Digital Logic

Ripple Carry Adder



$$2 + 2 = 4 ?$$

Digital Logic

Radix complements

Can we define negative numbers such that our adder still works?

$$x - x = 0$$

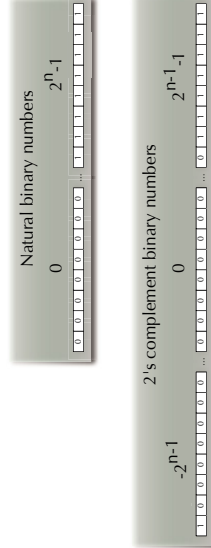
Or: what can you add to 42 in an 8bit binary representation such that the result will be 2^8 (and hence 0 in 8bits)?

$$\begin{array}{r}
 000101010110 \\
 + \\
 11101011110 \\
 = \\
 100000000000
 \end{array}$$

Digital Logic

2's complements

The 2's complement encoding interprets the natural binary range $2^{n-1} \dots 2^n - 1$ as negative numbers $-2^{n-1} \dots -1$



Digital Logic

2's complements

The 2's complement encoding interprets the natural binary range $2^{n-1} \dots 2^n - 1$ as negative numbers $-2^{n-1} \dots -1$

It's all in your mind!

Natural binary numbers															
2^{n-1}															
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

2's complement binary numbers															
2^{n-1}															
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

© 2019 Peter K. Zimmer, The Australian National University

page 53 of 81 | Chapter 1: "Digital Logic" - up to page 80

Digital Logic

Ripple Carry Adder

$2 - 1 = 1 ?$

© 2019 Peter K. Zimmer, The Australian National University

page 54 of 81 | Chapter 1: "Digital Logic" - up to page 80

Digital Logic

Ripple Carry Adder

$2 - 1 = 1 !$

... with an overall carry-flag indicated.

© 2019 Peter K. Zimmer, The Australian National University

page 55 of 81 | Chapter 1: "Digital Logic" - up to page 80

Digital Logic

Ripple Carry Adder

$2 - 1 = 1 ?$

© 2019 Peter K. Zimmer, The Australian National University

page 56 of 81 | Chapter 1: "Digital Logic" - up to page 80

Digital Logic

Ripple Carry Adder

$2 - 1 = 1 ?$

© 2019 Peter K. Zimmer, The Australian National University

page 57 of 81 | Chapter 1: "Digital Logic" - up to page 80

Digital Logic

Arithmetic Logic Unit (ALU)

A simple ALU which can ADD, XOR, AND, OR two arguments.

© 2019 Peter K. Zimmer, The Australian National University

page 58 of 81 | Chapter 1: "Digital Logic" - up to page 80

Digital Logic

Towards States

(everything up to here was combinational logic)

How do we make operations depends on:

- ⊞ ... an overflow in the previous operation?
- ⊞ ... the state of the CPU?
- ⊞ ... a counter having reached zero?
- ⊞ ... two arguments having been equal?
- ⊞ ... etc. pp.

⊞ We need to hold on to some states!

© 2019 Peter K. Zimmer, The Australian National University

page 59 of 81 | Chapter 1: "Digital Logic" - up to page 80

Digital Logic

States

© 2019 Peter K. Zimmer, The Australian National University

page 60 of 81 | Chapter 1: "Digital Logic" - up to page 80

Digital Logic

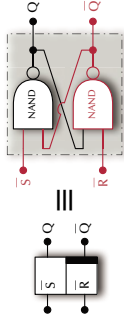
States

© 2019 Peter K. Zimmer, The Australian National University

page 61 of 81 | Chapter 1: "Digital Logic" - up to page 80

Digital Logic

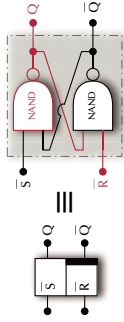
States



$\bar{S}$	$\bar{R}$	Q	$\bar{Q}$
0	0	?	?
0	1	?	?
1	0	?	?
1	1	Q	$\bar{Q}$

Digital Logic

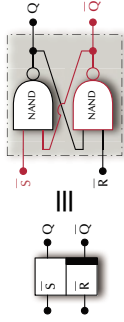
States



$\bar{S}$	$\bar{R}$	Q	$\bar{Q}$
0	0	?	?
0	1	1	0
1	0	?	?
1	1	Q	$\bar{Q}$

Digital Logic

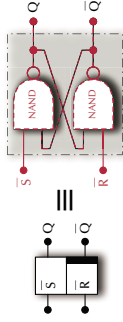
States



$\bar{S}$	$\bar{R}$	Q	$\bar{Q}$
0	0	?	?
0	1	1	0
1	0	0	1
1	1	Q	$\bar{Q}$

Digital Logic

States



$\bar{S}$	$\bar{R}$	Q	$\bar{Q}$
0	0	1/2	1/2
0	1	1	0
1	0	0	1
1	1	Q	$\bar{Q}$

Assuming Q as well as $\bar{Q}$ to be active simultaneously may lead to instability.

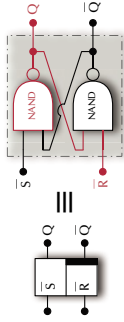
Digital Logic

Deriving SR Flip Flops

$\bar{S}$	$\bar{R}$	Q	$\bar{Q}$	Q minterms
0	0	0	1	$\bar{S} \wedge \bar{R} \wedge \bar{Q}$
0	0	1	0	$\bar{S} \wedge \bar{R} \wedge Q$
0	1	0	1	$\bar{S} \wedge R \wedge \bar{Q}$
0	1	1	0	$\bar{S} \wedge R \wedge Q$
1	0	0	1	$S \wedge \bar{R} \wedge \bar{Q}$
1	0	1	0	$S \wedge \bar{R} \wedge Q$
1	1	0	1	$S \wedge R \wedge \bar{Q}$
1	1	1	0	$S \wedge R \wedge Q$

Digital Logic

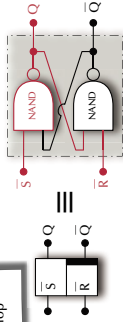
States



$\bar{S}$	$\bar{R}$	Q	$\bar{Q}$
0	0	?	?
0	1	1	0
1	0	?	?
1	1	Q	$\bar{Q}$

Digital Logic

States



$\bar{S}$	$\bar{R}$	Q	$\bar{Q}$
0	0	Forbidden	Forbidden
0	1	1	0
1	0	0	1
1	1	Q	$\bar{Q}$

Digital Logic

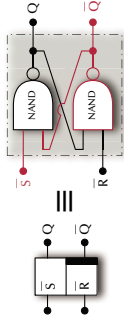
Deriving SR Flip Flops

$\bar{S}$	$\bar{R}$	Q	$\bar{Q}$	Q minterms
0	0	0	1	$\bar{S} \wedge \bar{R} \wedge \bar{Q}$
0	0	1	0	$\bar{S} \wedge \bar{R} \wedge Q$
0	1	0	1	$\bar{S} \wedge R \wedge \bar{Q}$
0	1	1	0	$\bar{S} \wedge R \wedge Q$
1	0	0	1	$S \wedge \bar{R} \wedge \bar{Q}$
1	0	1	0	$S \wedge \bar{R} \wedge Q$
1	1	0	1	$S \wedge R \wedge \bar{Q}$
1	1	1	0	$S \wedge R \wedge Q$

$$\text{NB: } Q = S \vee (R \wedge Q)$$

Digital Logic

States



$\bar{S}$	$\bar{R}$	Q	$\bar{Q}$
0	0	?	?
0	1	1	0
1	0	0	1
1	1	Q	$\bar{Q}$

Digital Logic

Deriving SR Flip Flops

$\bar{S}$	$\bar{R}$	Q	$\bar{Q}$
0	0	?	?
0	1	1	0
0	1	1	0
1	0	0	1
1	0	0	1
1	1	Q	$\bar{Q}$
1	1	Q	$\bar{Q}$

Digital Logic

Deriving SR Flip Flops

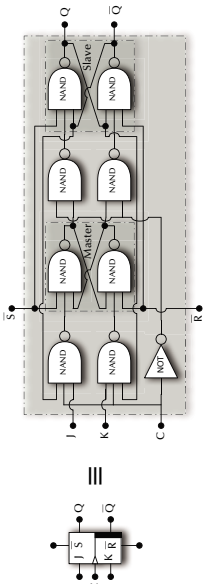
$\bar{S}$	$\bar{R}$	Q	$\bar{Q}$	Q minterms
0	0	0	1	$\bar{S} \wedge \bar{R} \wedge \bar{Q}$
0	0	1	0	$\bar{S} \wedge \bar{R} \wedge Q$
0	1	0	1	$\bar{S} \wedge R \wedge \bar{Q}$
0	1	1	0	$\bar{S} \wedge R \wedge Q$
1	0	0	1	$S \wedge \bar{R} \wedge \bar{Q}$
1	0	1	0	$S \wedge \bar{R} \wedge Q$
1	1	0	1	$S \wedge R \wedge \bar{Q}$
1	1	1	0	$S \wedge R \wedge Q$

$$\text{NB: } Q = S \vee (R \wedge Q) = \bar{S} \wedge \bar{R} \wedge Q$$



Digital Logic

Master-Slave JK Flip-Flop

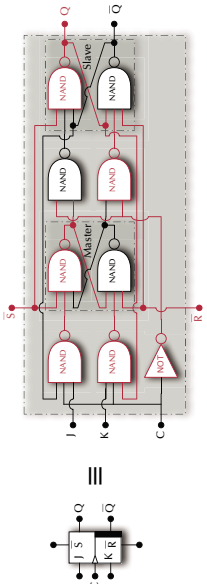


© 2019 Law R. Zimmer, The Australian National University
page 62 of 88 | Chapter 1: "Digital Logic" - up to page 80



Digital Logic

Master-Slave JK Flip-Flop

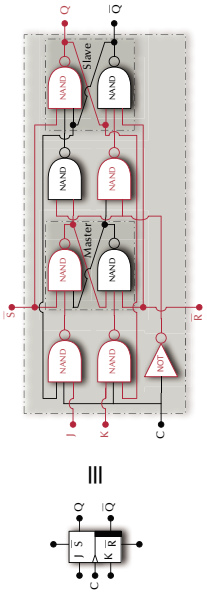


© 2019 Law R. Zimmer, The Australian National University
page 63 of 88 | Chapter 1: "Digital Logic" - up to page 80



Digital Logic

Master-Slave JK Flip-Flop

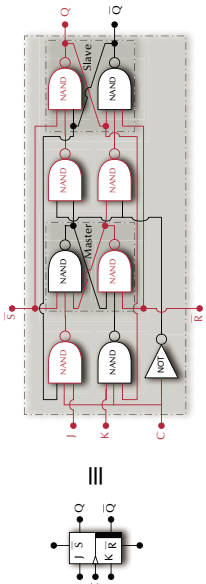


© 2019 Law R. Zimmer, The Australian National University
page 64 of 88 | Chapter 1: "Digital Logic" - up to page 80



Digital Logic

Master-Slave JK Flip-Flop



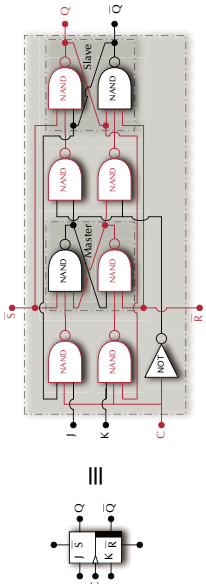
Master is reset on the rising clock edge

© 2019 Law R. Zimmer, The Australian National University
page 65 of 88 | Chapter 1: "Digital Logic" - up to page 80



Digital Logic

Master-Slave JK Flip-Flop

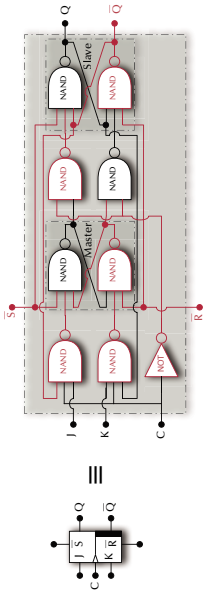


© 2019 Law R. Zimmer, The Australian National University
page 66 of 88 | Chapter 1: "Digital Logic" - up to page 80



Digital Logic

Master-Slave JK Flip-Flop



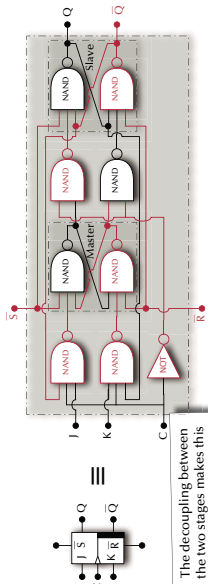
Slave follows on the falling clock edge

© 2019 Law R. Zimmer, The Australian National University
page 67 of 88 | Chapter 1: "Digital Logic" - up to page 80



Digital Logic

Master-Slave JK Flip-Flop



The decoupling between the two stages makes this flip-flop race free – even in JK-toggle mode.

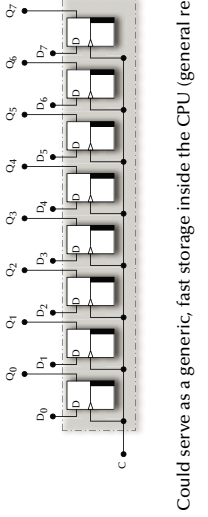
Slave follows on the falling clock edge

© 2019 Law R. Zimmer, The Australian National University
page 68 of 88 | Chapter 1: "Digital Logic" - up to page 80



Digital Logic

Register



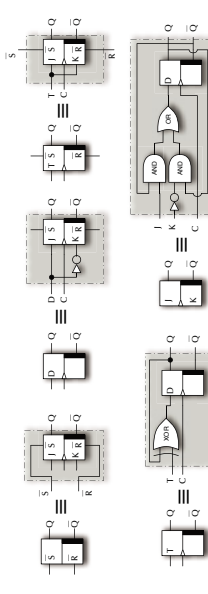
Could serve as a generic, fast storage inside the CPU (general register)
Or to hold internal states (e.g. ALU overflow) of the CPU
which are used by e.g. branching instructions.

© 2019 Law R. Zimmer, The Australian National University
page 69 of 88 | Chapter 1: "Digital Logic" - up to page 80



Digital Logic

Toggle Flip-Flop

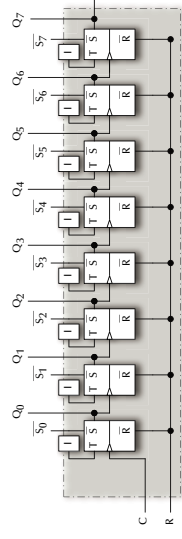


Toggle Flip-Flops change state with every clock cycle.

© 2019 Law R. Zimmer, The Australian National University
page 70 of 88 | Chapter 1: "Digital Logic" - up to page 80

Digital Logic

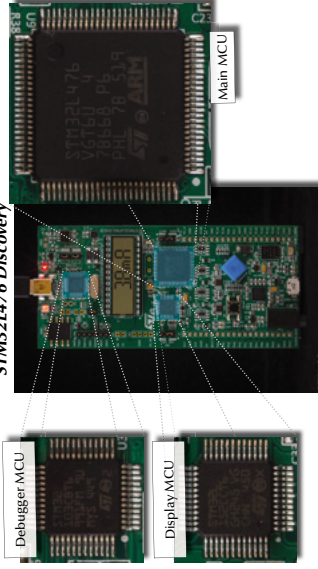
Counter



Your controller has many counters which can e.g. be used to delay operations without the need to execute instructions.

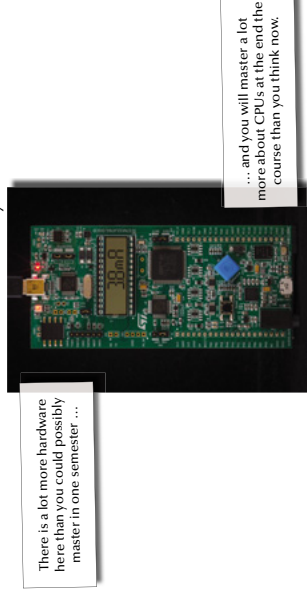
Digital Logic

STM32L476 Discovery



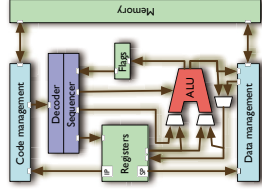
Digital Logic

STM32L476 Discovery



Digital Logic

Simple CPU Architecture



You can already build most of the components of a CPU by now.

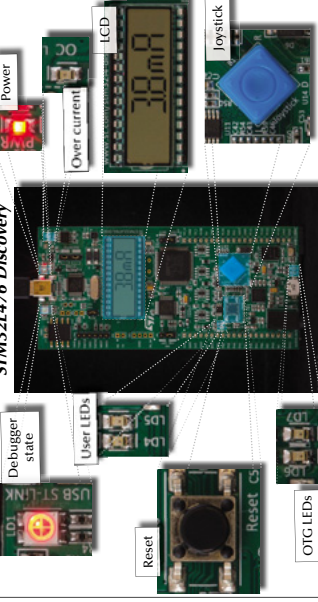
(The most essential missing component is the **sequencer** which is a specialized state-machine.)

We will come back to the CPU architectures towards the end of the course.

⚡ The next chapter will be about programming a CPU at machine level.

Digital Logic

STM32L476 Discovery



Digital Logic

Summary

Digital Logic

- **Boolean Algebra**
 - Truth tables and Boolean operations
 - Minterms and simplifying expressions
- **Combinational Logic**
 - Logic gates
 - Numbers
 - Adders, ALU
- **State-oriented Logic**
 - Flip-Flops, registers and counters
- **CPU Architecture**

Digital Logic

STM32L476 Discovery



Digital Logic

STM32L476 Discovery

